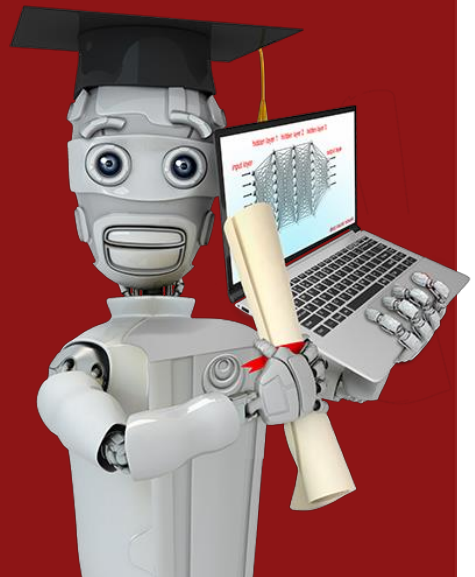
Copyright Notice

DeepLearning.AI

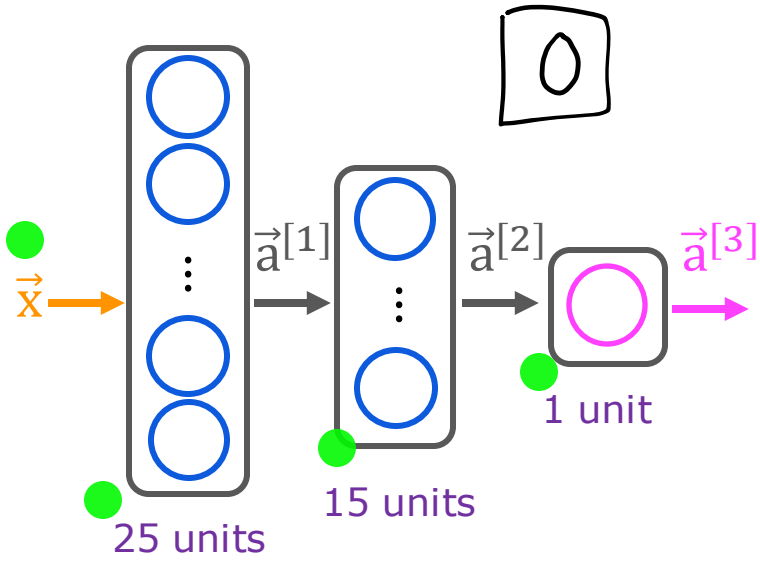
Stanford
ONLINE



Neural Network Training

TensorFlow implementation

Train a Neural Network in TensorFlow




```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
   model = Sequential([
     Dense(units=25, activation='sigmoid')
     Dense(units=15, activation='sigmoid')
     Dense(units=1, activation='sigmoid')
                              )]
from tensorflow.keras.losses import
BinaryCrossentropy
   model.compile(loss=BinaryCrossentropy())

   model.fit(X,Y,epochs=100)
```

0 1 2 3

epochs: number of steps in gradient descent

Given set of (x,y) examples

How to build and train this in code?

DeepLearning.AI

Neural Network Training

The Core Framework: The Three-Step Process

Both logistic regression and neural networks are trained using the same fundamental three-step process. Understanding this parallel is key to mastering the intuition.

1. **Specify the Model (Forward Propagation):** This defines how the output is calculated given an input `x` and the model's parameters (weights `w` and biases `b`).
2. **Define the Loss and Cost Function:** This defines *how* the model's performance will be measured.
3. **Minimize the Cost Function (Gradient Descent):** This is the optimization process that finds the best parameters to fit your data.

Let's dive deep into each step in the context of neural networks and TensorFlow.

Training Details

Model Training Steps

TensorFlow

1. specify how to compute output given input x and parameters w,b (define model)

$f_{\vec{w},b}(\vec{x}) = ?$

2. specify loss and cost

$L\left(f_{\vec{w},b}(\vec{x}), y\right)$ 1 example

$$J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m} L\left(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}\right)$$

3. Train on data to minimize $J(\vec{w}, b)$

logistic regression

```
z = np.dot(w,x) + b

f_x = 1/(1+np.exp(-z))
```

logistic loss

```
loss = -y * np.log(f_x)
   -(1-y) * np.log(1-f_x)
```

```
w = w - alpha * dj_dw
b = b - alpha * dj_db
```

neural network

```
model = Sequential([
          Dense(...)
          Dense(...)
          Dense(...)
                    ])
```

binary cross entropy

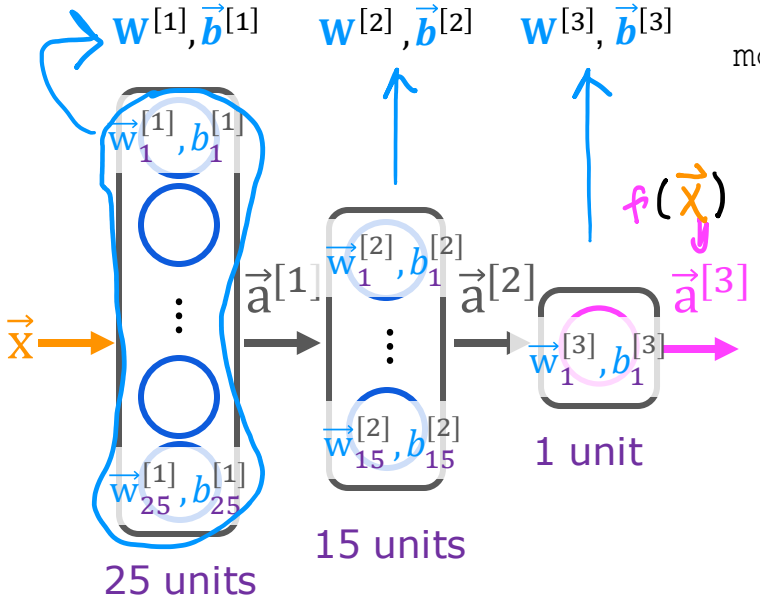
```
model.compile(
loss=BinaryCrossentropy())
```

```
model.fit(X,y,epochs=100)
```

1. Create the model

define the model

$f(\vec{x}) = ?$




```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
  Dense(units=25, activation='sigmoid')
  Dense(units=15, activation='sigmoid')
  Dense(units=1, activation='sigmoid')
                                    ])
```

Master's Insight #1: Why Columns and Not Rows?

Andrew explicitly stacks weights as **columns** in `W`. Why?
Because in linear algebra, the forward pass for a **batch** of data is:

$$Z = A_{in} \cdot W + b$$

- A_{in} shape: `(batch_size, num_inputs)`
- W shape: `(num_inputs, num_units)`
- Result Z shape: `(batch_size, num_units)`

By keeping `W` as `(inputs, units)`, we set up the matrix multiplication perfectly. This is exactly how TensorFlow's `Dense` layer stores its kernel (`W` is `(input_dim, units)`).

2. Loss and cost functions

Mnist digit classification problem

binary classification

$$L(f(\vec{x}), y) = -y\log(f(\vec{x})) - (1-y)\log(1-f(\vec{x}))$$

Compare prediction vs. target

logistic loss also known as binary cross entropy

```
model.compile(loss= BinaryCrossentropy())
```

regression (predicting numbers and not categories)

mean squared error

```
model.compile(loss= MeanSquaredError())
```

$$J(\mathbf{W}, \mathbf{B}) = \frac{1}{m}\sum_{i=1}^{m} L\left(f\left(\vec{x}^{(i)}\right), y^{(i)}\right)$$

$\mathbf{W}^{[1]}, \mathbf{W}^{[2]}, \mathbf{W}^{[3]}$ $\vec{b}^{[1]}, \vec{b}^{[2]}, \vec{b}^{[3]}$

$f_{W,B}(\vec{x})$

- **Loss Function (`L`)**: The error on a *single* training example `(x, y)`.
- **Cost Function (`J`)**: The average of the loss function over the *entire* training set.

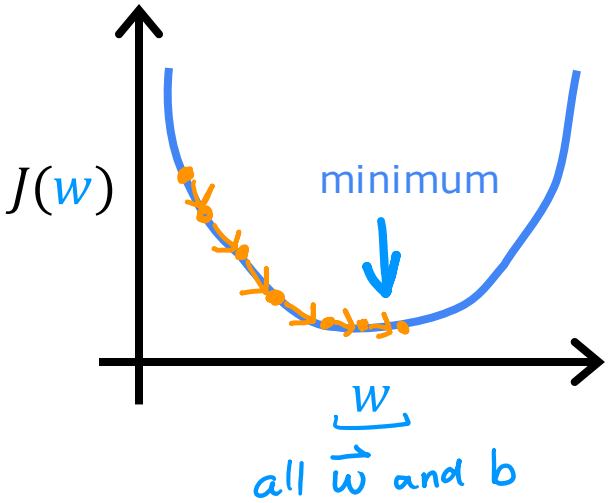
`J(w,b) = (1/m) * Σ L(f(x^i), y^i)`

```
from tensorflow.keras.losses import
  BinaryCrossentropy
```

Keras

```
from tensorflow.keras.losses import
  MeanSquaredError
```

3. Gradient descent



repeat {

$$w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

}

Compute derivatives for gradient descent using "back propagation"

2. **Backpropagation:** This is the key algorithm that TensorFlow uses to efficiently calculate those partial derivatives `(∂J/∂w)` and `(∂J/∂b)`. The name is derived from the fact that it propagates the error *backwards* through the network.
   - It starts at the output layer, calculates the error, and then moves backward, layer by layer, to compute the contribution of each weight and bias to the overall error.

```
model.fit(X,y,epochs=100)
```

1. **Gradient Descent:** It's an iterative algorithm that updates the parameters to move towards the minimum of the cost function.
   - `w = w - α * (∂J / ∂w)`
   - `b = b - α * (∂J / ∂b)`
   - Here, `α` is the learning rate.

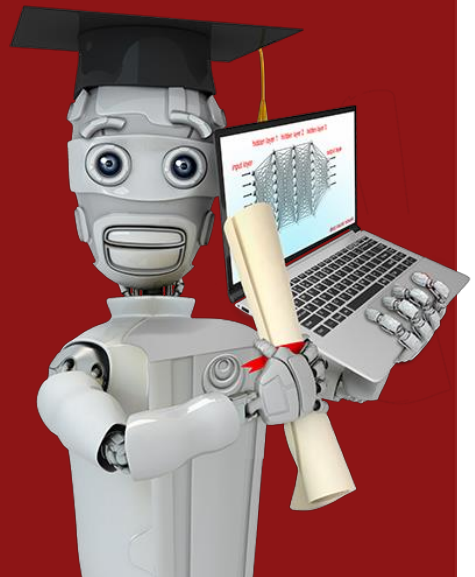
Neural network libraries

Use code libraries instead of coding ”from scratch”

Good to understand the implementation
(for tuning and debugging).

DeepLearning.AI

Stanford
ONLINE



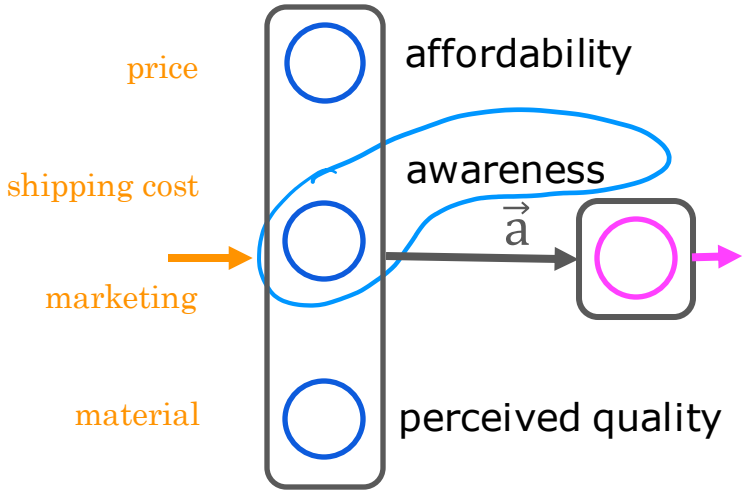
A Note on Activation Functions

The lectures mention that replacing the sigmoid activation function can make your neural network work much better. This is a crucial point for future weeks. Sigmoid is great for the output layer in binary classification (because it outputs a value between 0 and 1), but for hidden layers, it can cause problems like the "vanishing gradient" problem. **Rectified Linear Unit (ReLU)** is often a much better choice for hidden layers because it's faster and avoids this issue.

Activation Functions

Alternatives to the sigmoid activation

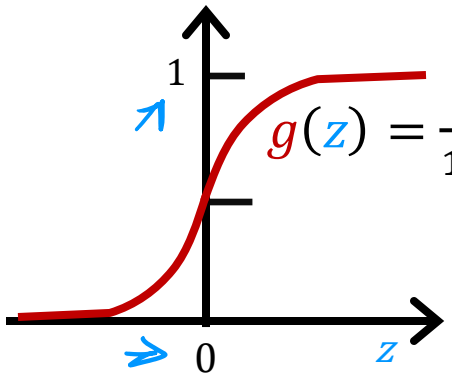
Demand Prediction Example



$$a_2^{[1]} = g(\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]})$$

z

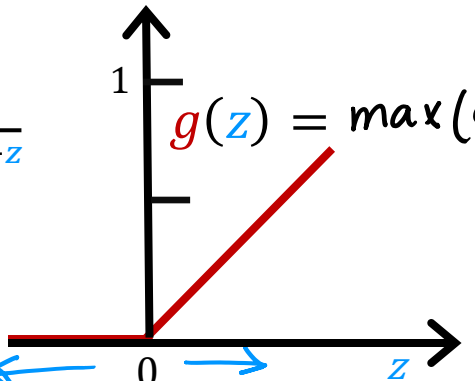
Sigmoid



$$g(z) = \frac{1}{1+e^{-z}}$$

$$0 < g(z) < 1$$

ReLU Rectified Linear Unit



$$g(z) = \max(0, z)$$

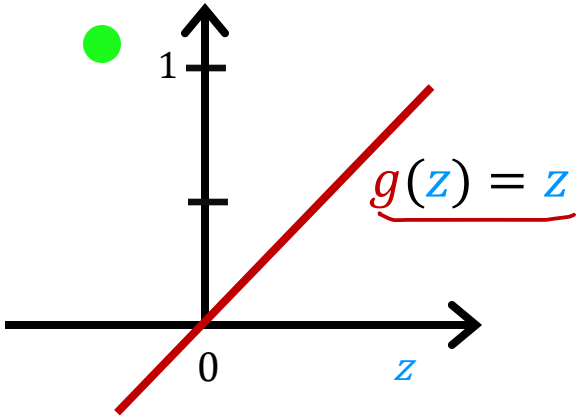
if $z < 0$, $g(z)$ is 0

if $z \geq 0$, $g(z)$ is z

Examples of Activation Functions

"No activation function"

Linear activation function



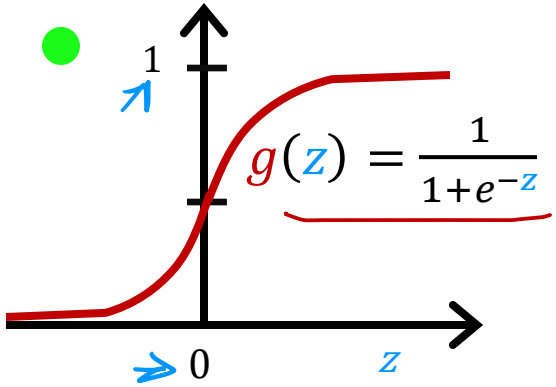
$a = g(z) = \vec{w} \cdot \vec{x} + b$

z

$a_2^{[1]} = g(\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]})$

z

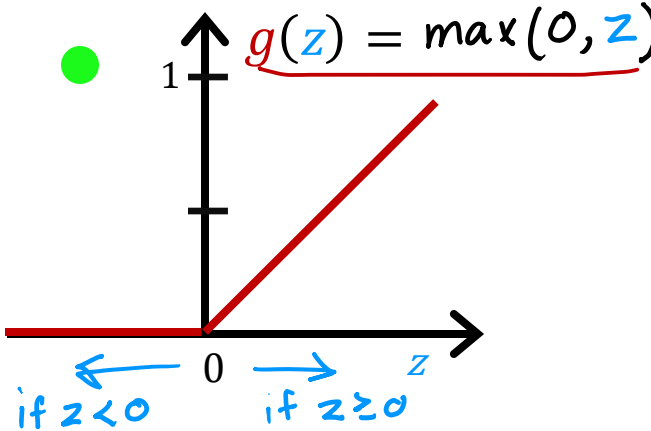
Sigmoid



$0 < g(z) < 1$

Later: softmax activation

ReLU Rectified Linear Unit

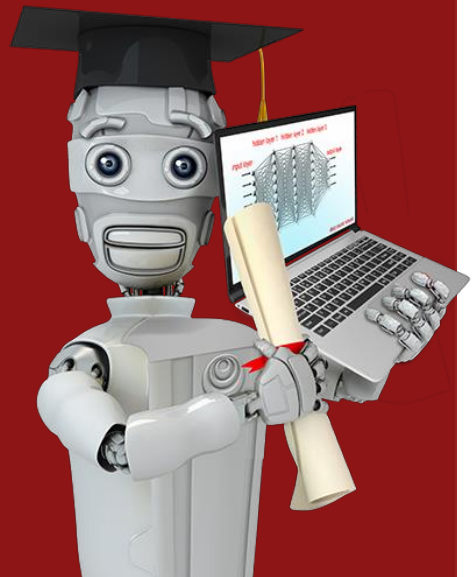


if $z < 0$ $g(z)$ is 0

if $z \geq 0$ $g(z)$ is z

DeepLearning.AI

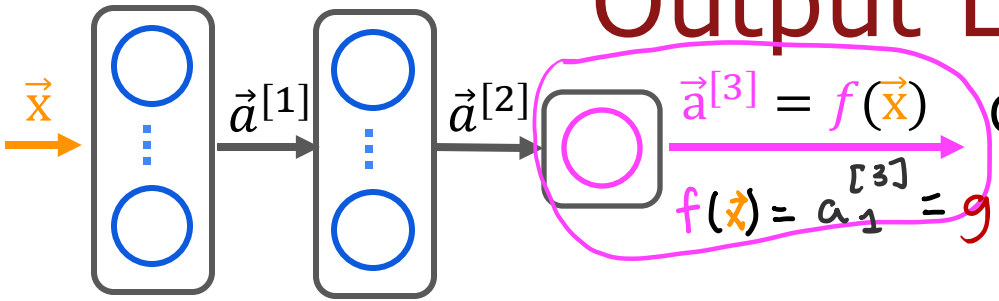
Stanford ONLINE



Activation Functions

Choosing activation functions

Output Layer

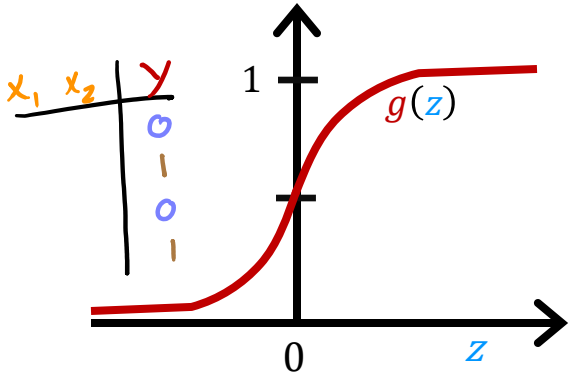


Choosing $g(z)$ for output layer?

Binary classification

Sigmoid

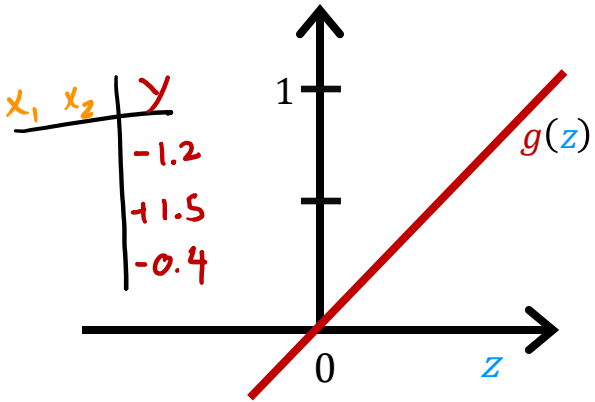
y=0/1



Regression

Linear activation function

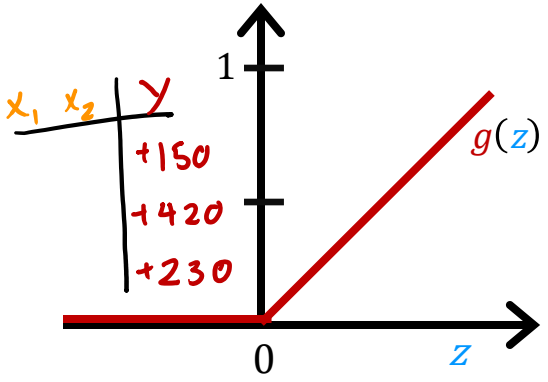
y = +/-



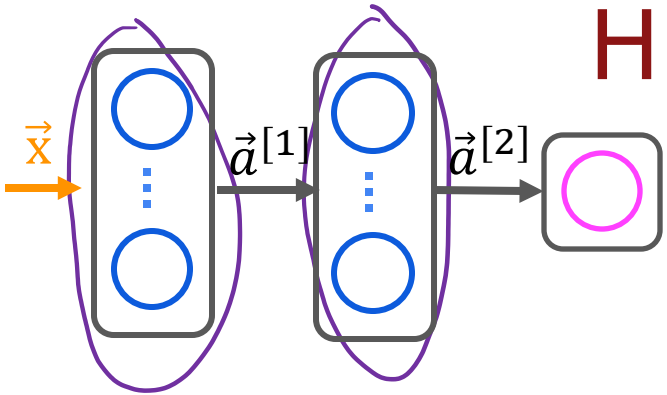
Regression

ReLU

Y = 0 or +



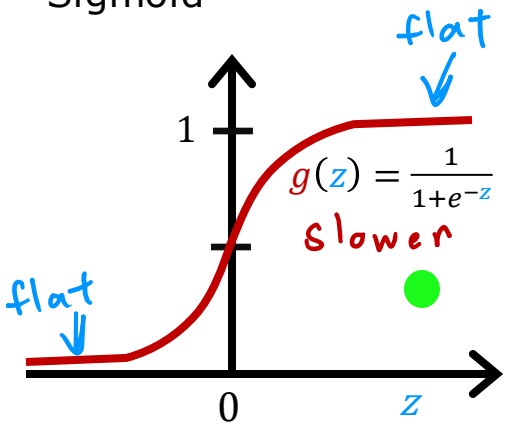
Hidden Layer

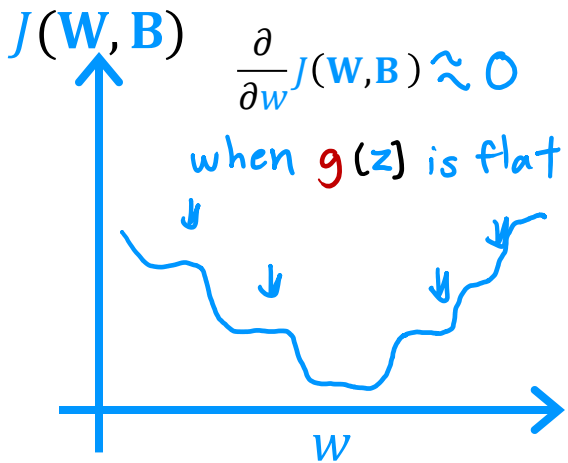


$\vec{a}^{[3]} = f(\vec{x})$

Choosing $g(z)$ for hidden layer

Sigmoid

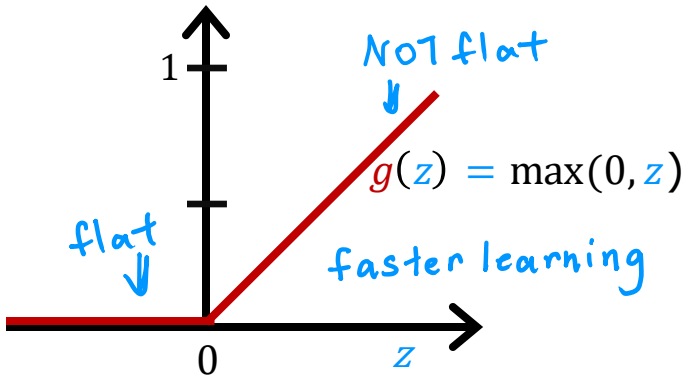




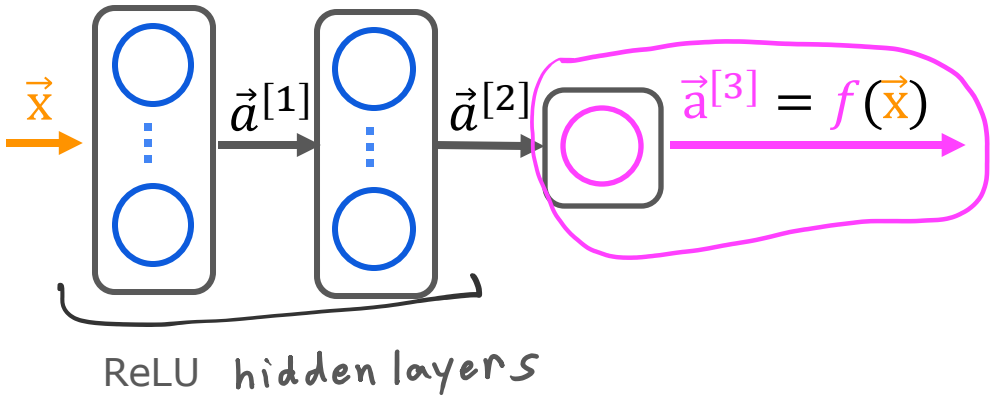
most common choice

ReLU

faster



Choosing Activation Summary



ReLU hidden layers

binary classification

activation='sigmoid'

regression y negative / positive

activation='linear'

regression $y \geq 0$

activation='relu'

```
from tf.keras.layers import Dense
model = Sequential([
  Dense(units=25, activation='relu'),    layer1
  Dense(units=15, activation='relu'),    layer2
  Dense(units=1,  activation='sigmoid')  layer3
])
```

or 'linear'
or 'relu'

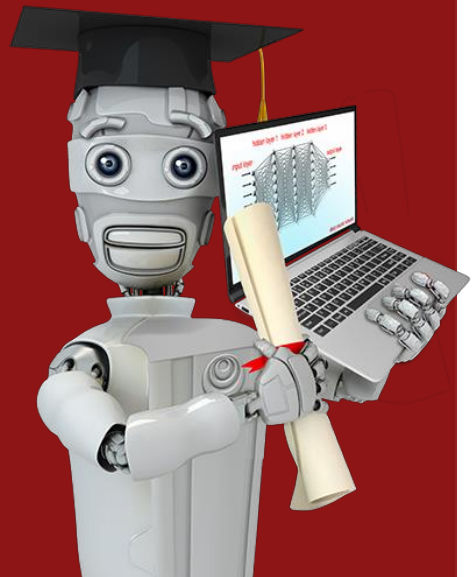
2. The Evolutionary Timeline of Activations

To fix the mess that ReLU left behind, researchers created the next generations of activation functions that you brought up earlier:

```
Sigmoid / Tanh
   |
   ▼ (Fixes Vanishing Gradient)
ReLU (But introduces Dying ReLU)
   |
   ├→ Leaky ReLU / PReLU (Fixes Dying ReLU with a small slope like 0.01x
   |
   ▼ (Fixes "sharp corners" for massive models)
GELU / SwiGLU (Smooth, gated curves used in modern LLMs)
```

DeepLearning.AI

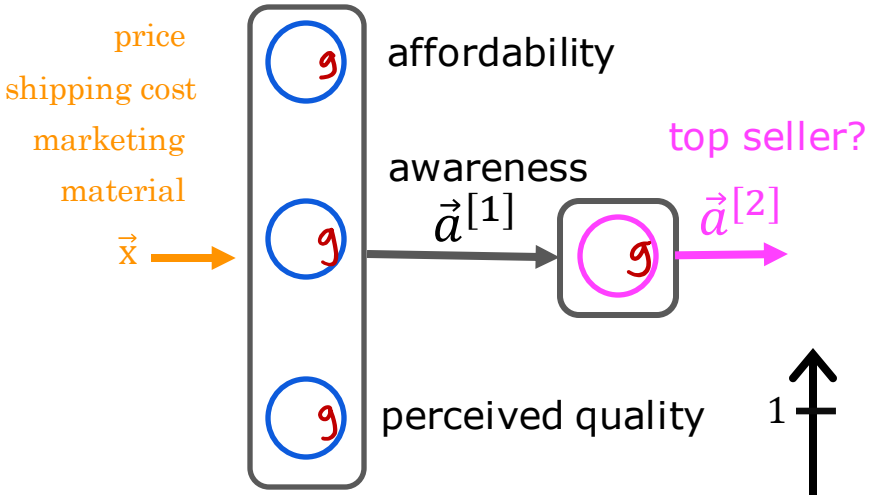
Stanford
ONLINE

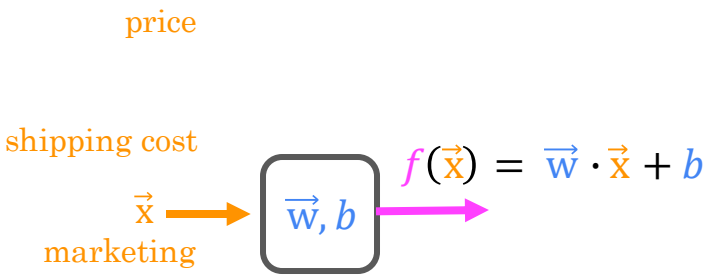


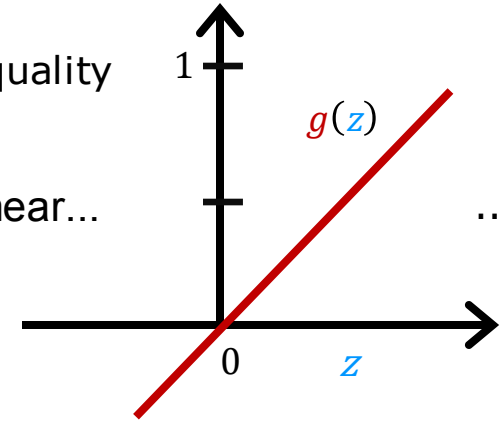
Activation Functions

Why do we need activation functions?

Why do we need activation functions?



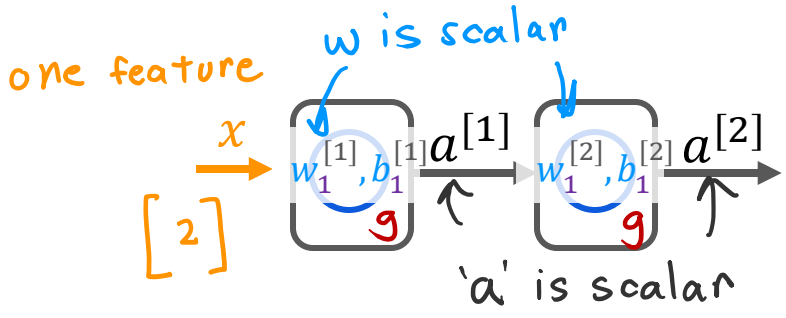




if all $g(z)$ are linear...

...no different than linear regression

Linear Example



$$g(z) = z$$

$$a^{[1]} = w_1^{[1]} x + b_1^{[1]}$$

$$a^{[2]} = w_1^{[2]} a^{[1]} + b_1^{[2]}$$

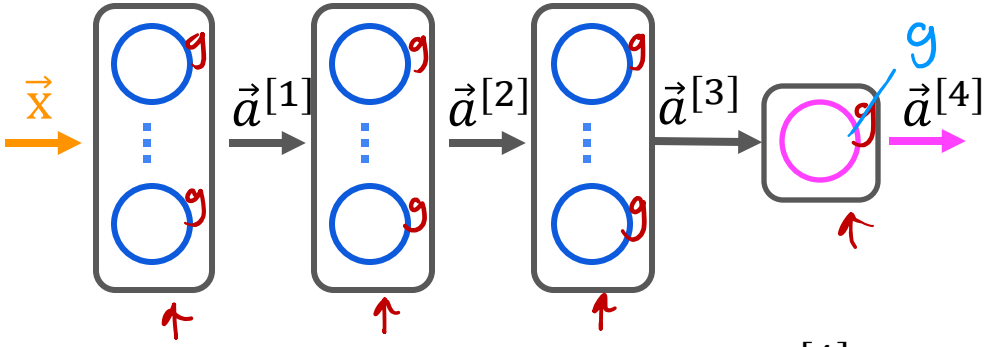
$$= w_1^{[2]} (w_1^{[1]} x + b_1^{[1]}) + b_1^{[2]}$$

$$\vec{a}^{[2]} = \underbrace{(\vec{w}_1^{[2]} \vec{w}_1^{[1]})}_{w} x + \underbrace{w_1^{[2]} b_1^{[1]} + b_1^{[2]}}_{b}$$

$$\vec{a}^{[2]} = w x + b$$

$f(x) = wx + b$ linear regression

Example



$g(z) = z$

$$\vec{a}^{[4]} = \vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]}$$

all linear (including output)
↳ equivalent to linear regression

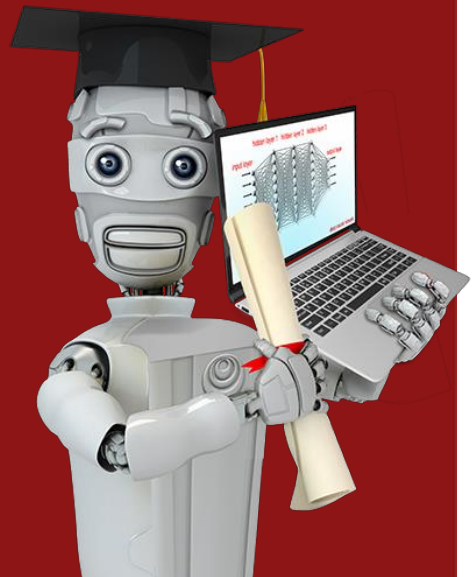
$$\vec{a}^{[4]} = \frac{1}{1+e^{-(\vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]})}}$$

output activation is sigmoid
(hidden layers still linear)
↳ equivalent to logistic regression

Don't use linear activations in hidden layers (use ReLU)

DeepLearning.AI

Stanford ONLINE



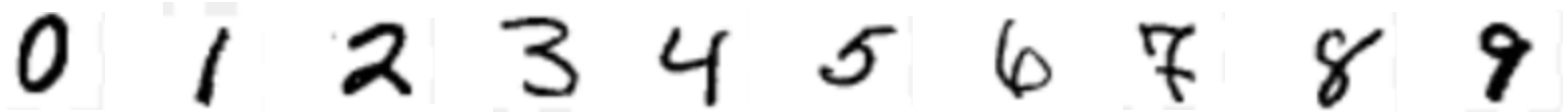
Multiclass Classification

Multiclass

1. The Problem Definition (Video 60)

- **Binary vs. Multi-class**: In binary, `y` is 0 or 1. In multi-class, `y` can take **more than 2** discrete values (e.g., digits 0-9, 4 types of diseases, or 4 types of product defects).
- **Decision Boundary**: Instead of a single line separating 2 regions, the algorithm learns boundaries that partition the feature space into **N distinct regions** (e.g., 4 regions for 4 classes).

MNIST example

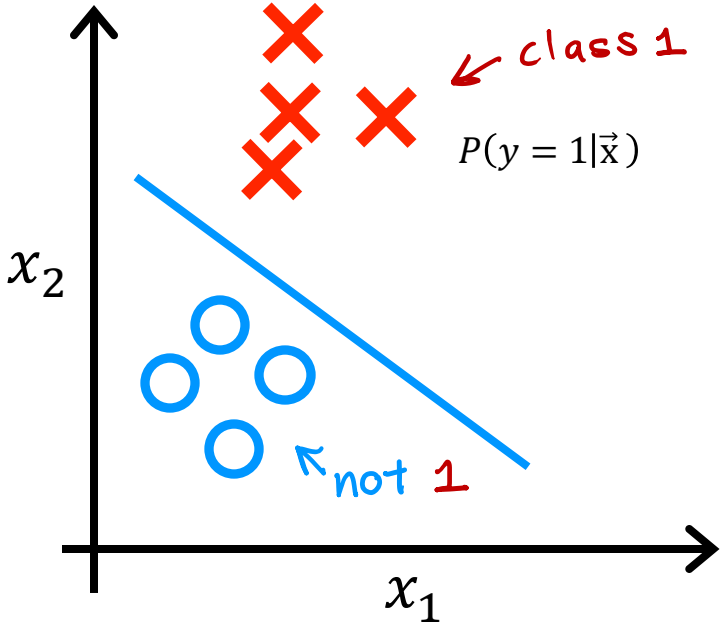


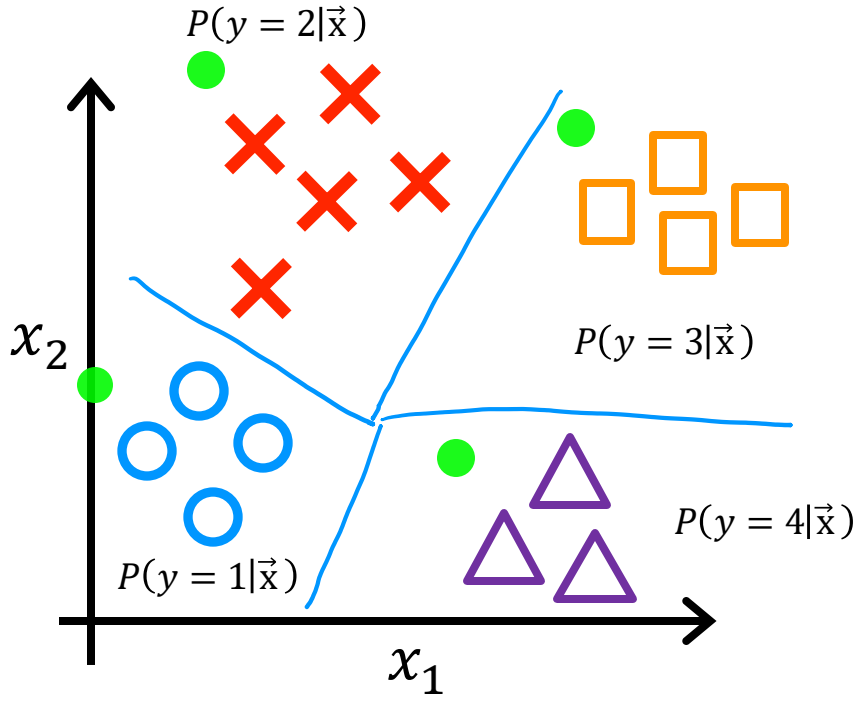
$y = 0$ 1 2 3 4 5 6 7 8 9

$y = 7$

multiclass classification problem:
target y can take on more than two possible values

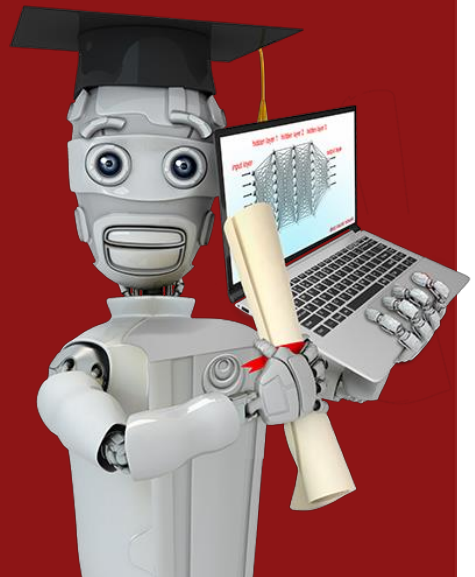
Multiclass classification example





DeepLearning.AI

Stanford ONLINE



Multiclass Classification

Softmax

- *Review*: Logistic regression computes `z = w·x + b`, then `a1 = sigmoid(z) = P(y=1)`. We also implicitly compute `a2 = 1 − a1 = P(y=0)`.

Logistic regression (2 possible output values)

$z = \vec{w} \cdot \vec{x} + b$

$a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x})$ 0.71

$a_2 = 1 - a_1 = P(y=0|\vec{x})$ 0.29

Softmax regression (N possible outputs) $y = 1, 2, 3, \ldots, N$

$z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j = 1, \ldots, N$

parameters $w_1, w_2, \ldots, w_N$ $b_1, b_2, \ldots, b_N$

$a_j = \frac{e^{z_j}}{\sum_{k=1}^{N} e^{z_k}} = P(y=j|\vec{x})$

note: $a_1 + a_2 + \ldots + a_N = 1$

Softmax regression (4 possible outputs) $y = 1, 2, 3, 4$

$z_1 = \vec{w}_1 \cdot \vec{x} + b_1$

$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=1|\vec{x})$ 0.30

$z_2 = \vec{w}_2 \cdot \vec{x} + b_2$

$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=2|\vec{x})$ 0.20

$z_3 = \vec{w}_3 \cdot \vec{x} + b_3$

$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=3|\vec{x})$ 0.15

$z_4 = \vec{w}_4 \cdot \vec{x} + b_4$

$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=4|\vec{x})$ 0.35

Cost

Logistic regression

$$z = \vec{w} \cdot \vec{x} + b$$

$$a_1 = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1|\vec{x})$$

$$a_2 = 1 - a_1 = P(y = 0|\vec{x})$$

$$loss = -y \log a_1 - (1 - y) \log(1 - a_1)$$

if $y = 1$ if $y = 0$

$$J(\vec{w}, b) = \text{average loss}$$

Softmax regression

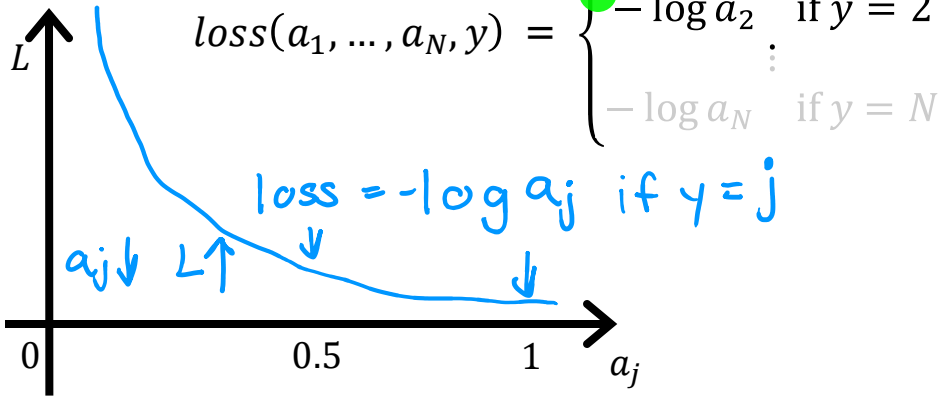
$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \cdots + e^{z_N}} = P(y = 1|\vec{x})$$

$$\vdots$$

$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \cdots + e^{z_N}} = P(y = N|\vec{x})$$

Crossentropy loss

$$loss(a_1, \ldots, a_N, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ -\log a_2 & \text{if } y = 2 \\ \vdots & \\ -\log a_N & \text{if } y = N \end{cases}$$



2. The Math: Generalizing Logistic Regression (Video 61)

- *Review*: Logistic regression computes `z = w·x + b`, then `a1 = sigmoid(z) = P(y=1)`. We also implicitly compute `a2 = 1 - a1 = P(y=0)`.
- *The Expansion (Softmax)*: For `N` classes, we compute **N separate linear functions**:
  - `z_1 = w_1·x + b_1`
  - `z_2 = w_2·x + b_2`
  - ...
  - `z_N = w_N·x + b_N`
- *Softmax Formula*: To turn these into probabilities, we apply:
  - `a_j = e^(z_j) / ( e^(z_1) + e^(z_2) + ... + e^(z_N) )`
- **Interpretation**: `a_j` = The model's estimated probability that `y = j`, given the input features `x`.
- **Key Property**: By design, `a_1 + a_2 + ... + a_N = 1.0` (they always sum to exactly 1).

• Interpretation: a_j = The model's estimated

3. The Cost Function (Loss) for Softmax (Video 61)

- Andrew emphasizes we do not use squared error. We use the **Categorical Cross-Entropy Loss**.
- **The Rule**: If the true label for a training example is class `j` (e.g., `y = j`), the loss for that single example is:
  - **Loss =** `-log(a_j)`
- **Intuition**:
  - If the model is highly confident and correct (`a_j` close to 1.0) → `-log(1.0)` is near 0 (Low Loss).
  - If the model is uncertain or wrong (`a_j` is small, e.g., 0.1) → `-log(0.1)` is large (High Loss).
- **Overall Cost**: The total cost is the **average** of this loss over *all* training examples.

4. Connection to Neural Networks (Video 61)

- To use this in a Neural Network:
  - The **Output Layer** must have `N` units (one for each class).
  - Compute the raw logits `z` (using a linear activation).
  - The loss function handles the Softmax internally.

1. The Architecture Shift (From 2 to 10 Classes)

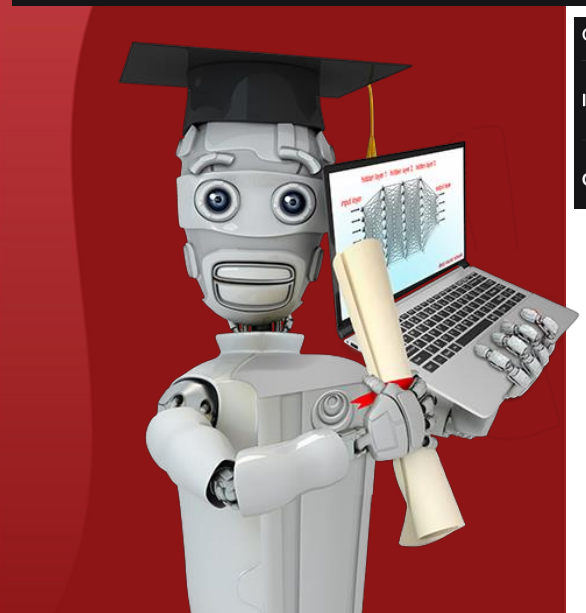
In Lecture 62, Andrew expands the architecture. For digit recognition (0–9), the output layer must have **10 units** instead of 1.

Critically, the **Softmax Activation** is fundamentally different from Sigmoid or ReLU:

- **Sigmoid/ReLU**: They are **element-wise** functions. To compute `a1`, you only look at `z1`. To compute `a2`, you only look at `z2`. They are independent.
- **Softmax**: It is a **vector-wise** function. To compute `a1` (the probability of digit 0), you must look at **all** `z1` through `z10`.
  `a1 = e^z1 / (e^z1 + e^z2 + ... + e^z10)`

This interdependence is what forces the outputs to sum to exactly `1.0`, creating a valid probability distribution.

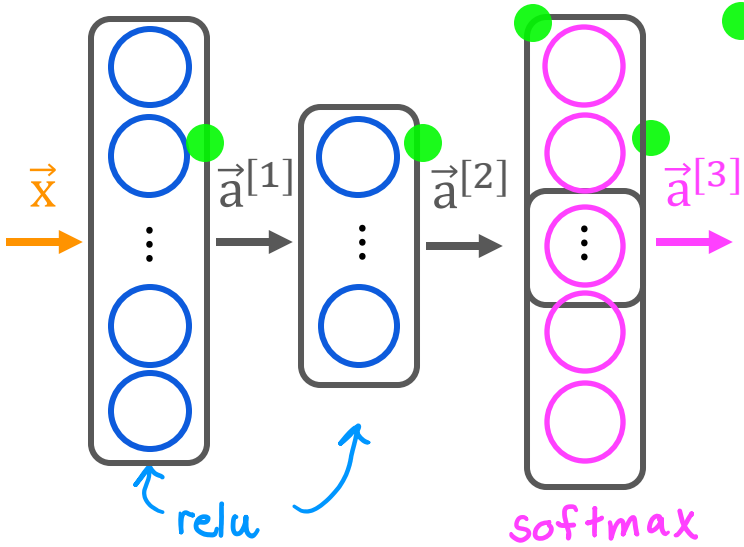
Multiclass Classification



Concept	Applied To	Why we do it	The Hack
Input Normalization	Raw features (Age, Income, Pixels)	To ensure smooth, fast Gradient Descent and prevent dead neurons.	Standardize to `(x - μ)/σ` before feeding into the model.
Output Logits Trick	Output layer (`z` values)	To prevent numerical underflow/overflow when computing `e^z` and `log`.	Set `activation='linear'` and `from_logits=True` in the loss.

Neural Network with Softmax output

Neural Network with Softmax output



$$z_1 = \vec{w}_1 \cdot \vec{a}^{[2]} + b_1 \qquad a_1 = \frac{e^{z_1}}{e^{z_1} + \cdots + e^{z_{10}}} = P(y = 1|\vec{x})$$

$$\vdots$$

$$z_{10} = \vec{w}_{10} \cdot \vec{a}^{[2]} + b_{10} \qquad a_{10} = \frac{e^{z_{10}}}{e^{z_1} + \cdots + e^{z_{10}}} = P(y = 10|\vec{x})$$

logistic regression

$$a_1^{[3]} = g\left(z_1^{[3]}\right) \quad a_2^{[3]} = g\left(z_2^{[3]}\right)$$

softmax

$$\vec{a}^{[3]} = \left(a_1^{[3]}, \ldots a_{10}^{[3]}\right) = g\left(z_1^{[3]}, \ldots, z_{10}^{[3]}\right)$$

MNIST with softmax

1. specify the model

$f_{\vec{w},b}(\vec{x}) = ?$

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
  Dense(units=25, activation='relu')
  Dense(units=15, activation='relu')
  Dense(units=10, activation='softmax')
                  )]
```

2. specify loss and cost

$L\left(f_{\vec{w},b}(\vec{x}), y\right)$

```
from tensorflow.keras.losses import
  SparseCategoricalCrossentropy
model.compile(loss= SparseCategoricalCrossentropy() )
```

3. Train on data to minimize $J(\vec{w}, b)$

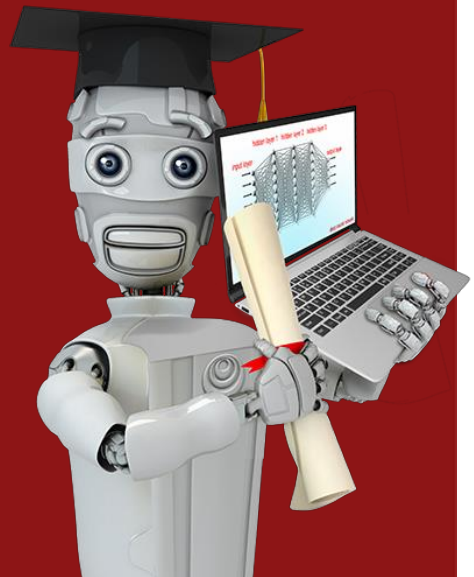
```
model.fit(X,Y,epochs=100)
```

Note: better (recommended) version later.

Don't use the version shown here!

DeepLearning.AI

Stanford
ONLINE



Multiclass Classification

Improved implementation of softmax

Numerical Roundoff Errors

option 1

$$x = \frac{2}{10{,}000}$$

option 2

$$x = \left(1 + \frac{1}{10{,}000}\right) - \left(1 - \frac{1}{10{,}000}\right) =$$

- **Option 1 (Safe):** `x = 1/20000` → The computer calculates it directly. Result: `0.00005`.
- **Option 2 (Dangerous):** `x = (1 + 1/100000) - (1 - 1/100000)` → Mathematically, this *should* equal `2/100000 = 1/20000`.
  - However, computers use **floating-point arithmetic** with finite memory (e.g., 32-bit or 64-bit).
  - `1 + 0.00001` is calculated, but the computer rounds it because it can't hold that many decimal places precisely. When you subtract `(1 - 0.00001)`, the tiny precision errors don't cancel out perfectly—they explode into massive rounding errors.

Now, apply this to Neural Networks:

The naive code computes:

1. `z` (logits) – which can be very large (e.g., `z = 10`) or very negative (`z = -10`).
2. `a = e^z / sum(e^z)` – If `z=10`, `e^10` is ~22,000. If `z=-10`, `e^-10` is ~0.000045. We are juggling insanely tiny and insanely huge numbers.
3. `Loss = -log(a)` – If `a` is slightly rounded to `0.0` due to underflow, then `-log(0)` equals **infinity**, breaking your entire gradient descent.

Numerical Roundoff Errors

More numerically accurate implementation of logistic loss:

Logistic regression:

$$a = g(z) = \frac{1}{1+e^{-z}}$$

Original loss

$$loss = -y\log(a) - (1-y)\log(1-a)$$

More accurate loss (in code)

$$loss = -y\log\left(\frac{1}{1+e^{-z}}\right) - (1-y)\log\left(1-\frac{1}{1+e^{-z}}\right)$$

```
model = Sequential([
    Dense(units=25, activation='relu')
    Dense(units=15, activation='relu')
    Dense(units=10, activation='sigmoid')
])
model.compile(loss=BinaryCrossEntropy() )
```

'linear'

```
model.compile(loss=BinaryCrossEntropy(from_logits=True) )
```

logit: z

Separate the Linear layer from the Softmax calculation.

Instead of asking TensorFlow to compute the intermediate `a` (the probability), you ask it to compute the raw `z` (logits) and combine the Softmax *inside* the loss function.

More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \ldots, a_{10}) = g(z_1, \ldots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots & \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
    Dense(units=25, activation='relu')
    Dense(units=15, activation='relu')
    Dense(units=10, activation='softmax')
```

'linear'

```
model.compile(loss=SparseCategoricalCrossEntropy() )
```

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \dfrac{e^{z_1}}{e^{z_1} + \cdots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots & \\ -\log \dfrac{e^{z_{10}}}{e^{z_1} + \cdots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

- In the first version, `model.predict(x)` gives you probabilities (e.g., `[0.1, 0.8, 0.05...]`), which are easy to interpret.
- In the "logits" version, `model.predict(x)` gives you raw `z` values (which can be negative or positive, arbitrary scales).

```
model.compile(loss=SparseCrossEntropy(from_logits=True) )
```

MNIST (more numerically accurate)

model

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
  Dense(units=25, activation='relu')
  Dense(units=15, activation='relu')
  Dense(units=10, activation='linear') )]
```

loss

```
from tensorflow.keras.losses import
  SparseCategoricalCrossentropy
model.compile(...,loss=SparseCategoricalCrossentropy(from_logits=True) )
```

fit

```
model.fit(X,Y,epochs=100)
```

predict

```
logits = model(X)
f_x = tf.nn.softmax(logits)
```

not $a_1 \ldots a_{10}$ is $z_1 \ldots z_{10}$

logistic regression (more numerically accurate)

model

```
model = Sequential([
  Dense(units=25, activation='sigmoid')
  Dense(units=15, activation='sigmoid')
  Dense(units=1, activation='linear')
from tensorflow.keras.losses import
  BinaryCrossentropy
```

loss

```
model.compile(..., BinaryCrossentropy(from_logits=True)) )
```

```
model.fit(X,Y,epochs=100)
```

fit

```
logit = model(X)
```

predict

```
f_x = tf.nn.sigmoid(logit)
```

When you use `from_logits=True`, TensorFlow doesn't force the output through a sigmoid before calculating the loss. It combines the sigmoid and the log-loss into one mathematical expression and simplifies it to avoid rounding errors.

```
# Output layer: NO activation function (just raw z values)
model.add(Dense(units=10, activation='linear'))

# Compile: Tell TensorFlow to apply softmax internally, but with numerical tricks
model.compile(loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

Why does this work mathematically?

Because TensorFlow uses the **Log-Sum-Exp trick.**

Look at the loss function mathematically. We want to compute:

`Loss = -log( e^z_j / Σ e^z_k )`

Using logarithm properties, we can simplify this *before* ever computing the exponentials:

`Loss = -[ z_j - log( Σ e^z_k ) ]`

`Loss = log( Σ e^z_k ) - z_j`

- **In the naive version**, we compute `e^z_j` (huge) and `Σ e^z_k` (huge), divide them (risking underflow/overflow), and *then* take the log.
- **In the "from_logits" version**, TensorFlow computes `log( Σ e^z_k )` directly using an optimized routine that "factors out" the maximum `z` value.

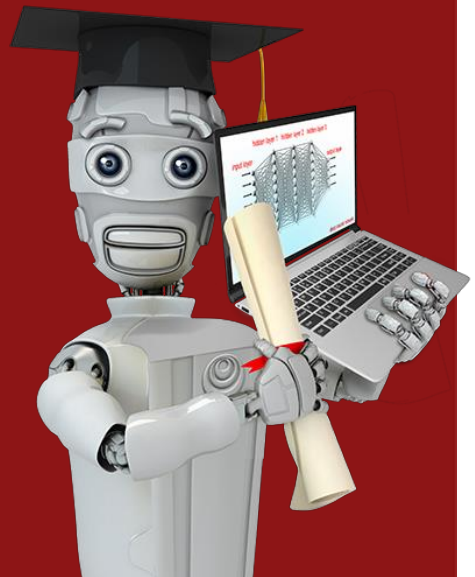
Here is the hidden trick TensorFlow uses internally:

1. Let `m = max(z_1, z_2, ..., z_N)`.
2. Compute `log( Σ e^z_k ) = m + log( Σ e^(z_k - m) )`.
3. Since `z_k - m` is always `≤ 0`, `e^(z_k - m)` is always between `0` and `1`. **No huge numbers!** The computation is perfectly numerically stable.

1. **Floating-Point is Fragile:** As a data scientist, you must always be suspicious of intermediate calculations involving exponentials (`e^x`) and logarithms (`log(x)`), especially when `x` approaches extreme values.
2. **The "Logits" Philosophy:** In deep learning frameworks, **"Logits"** are the raw, unnormalized outputs of the last linear layer. The framework's loss functions are mathematically designed to accept these logits and apply the activation function internally using high-precision, cancellation-proof algorithms.
3. **Best Practice:** In TensorFlow, unless you specifically need the probabilities for a lightweight inference deployment, **always use** `from_logits=True` in your loss function and set your output layer to `activation='linear'`. Your gradients will be more stable, your loss will converge smoothly, and you will avoid those dreaded `NaN` (Not a Number) losses that plague beginners.

DeepLearning.AI

Stanford
ONLINE



Multi-label Classification

Classification with multiple outputs (Optional)

Multi-label Classification

x

Is there a car?
Is there a bus?
Is there a pedestrian

yes
no
yes

$y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$

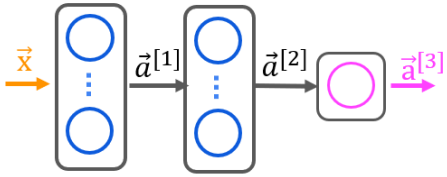
no
no
yes

$y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

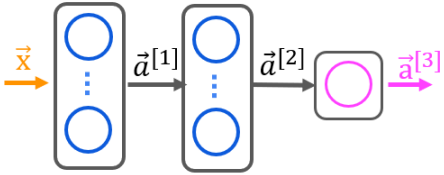
yes
yes
no

$y = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$

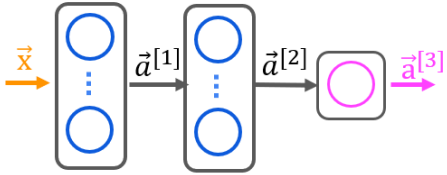
Multiple classes



car

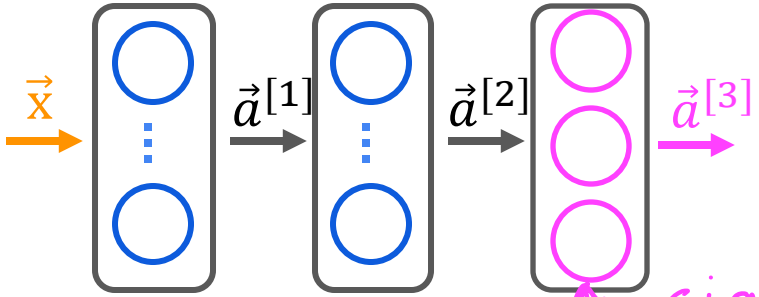


bus



pedestrian

Alternatively, train one neural network with three outputs

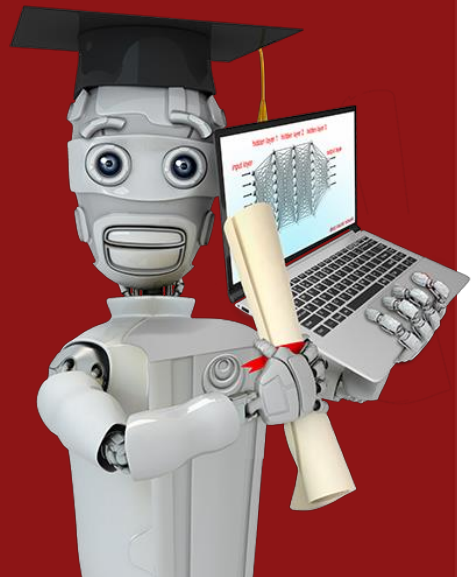


$$\vec{a}^{[3]} = \begin{bmatrix} a_1^{[3]} \\ a_2^{[3]} \\ a_3^{[3]} \end{bmatrix}$$

car
bus
pedestrian

DeepLearning.AI

Stanford
ONLINE



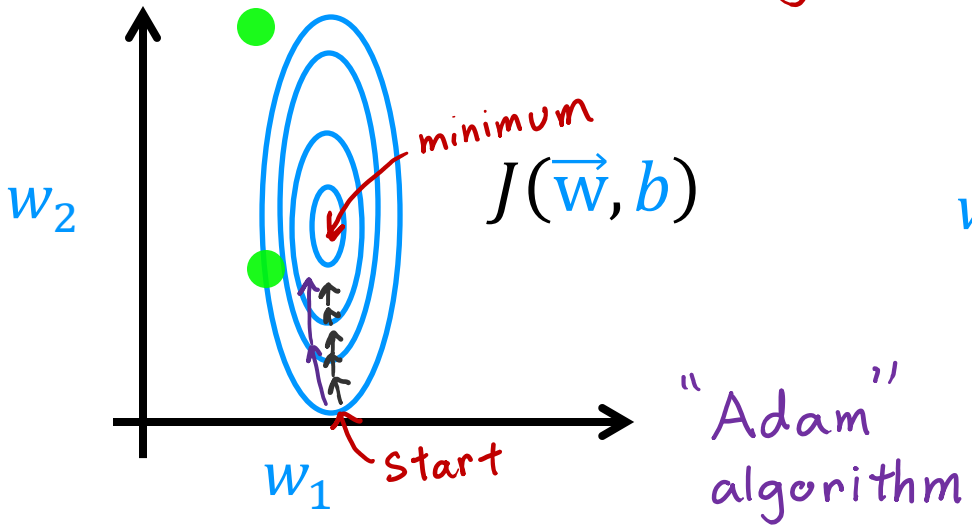
Additional Neural Network Concepts

Advanced Optimization

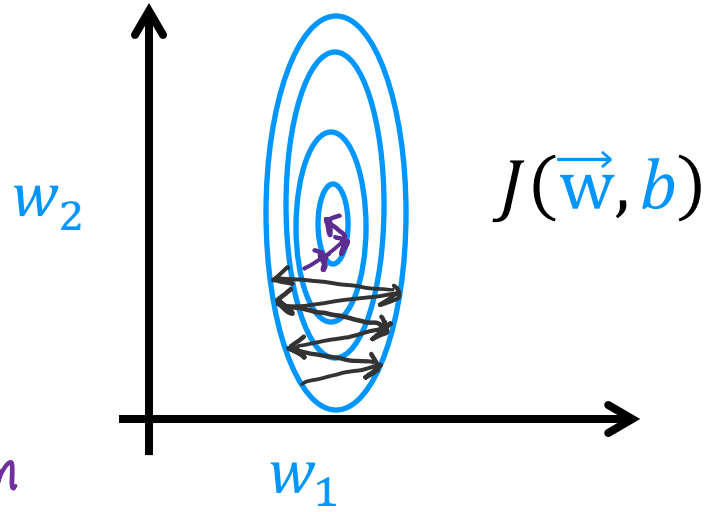
Gradient Descent

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

learning rate



"Adam" algorithm



Go faster – increase α

Go slower – decrease α

Adam Algorithm Intuition

Adam: Adaptive Moment estimation

not just one α

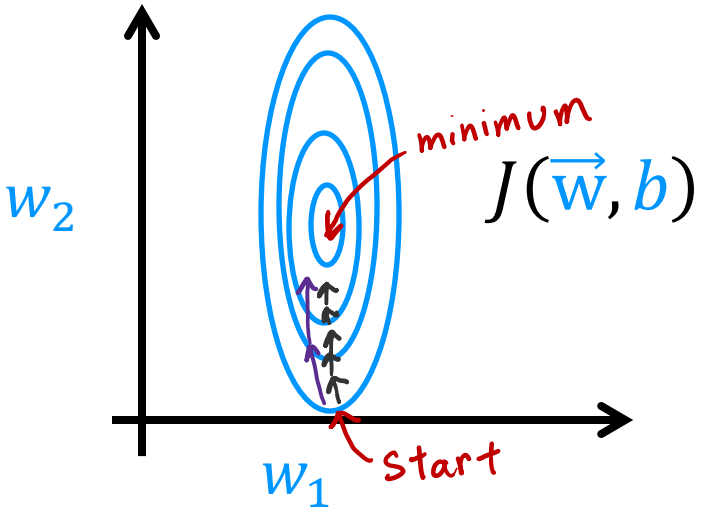
$$w_1 = w_1 - \alpha_1 \frac{\partial}{\partial w_1} J(\vec{w}, b)$$

$$\vdots$$

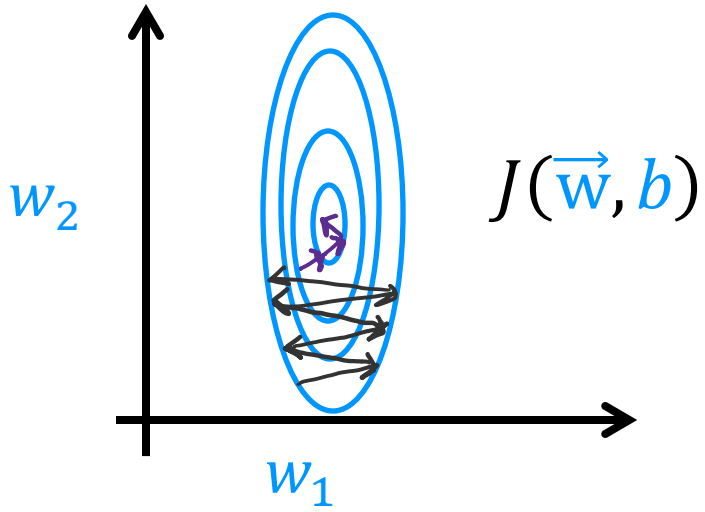
$$w_{10} = w_{10} - \alpha_{10} \frac{\partial}{\partial w_{10}} J(\vec{w}, b)$$

$$b = b - \alpha_{11} \frac{\partial}{\partial b} J(\vec{w}, b)$$

Adam Algorithm Intuition



If w_j (or b) keeps moving in same direction, increase α_j.



If w_j (or b) keeps oscillating, reduce α_j.

MNIST Adam

model

```
model = Sequential([
          tf.keras.layers.Dense(units=25, activation='sigmoid')
          tf.keras.layers.Dense(units=15, activation='sigmoid')
          tf.keras.layers.Dense(units=10, activation='linear')
])
```

compile

$\alpha = 10^{-3} = 0.001$

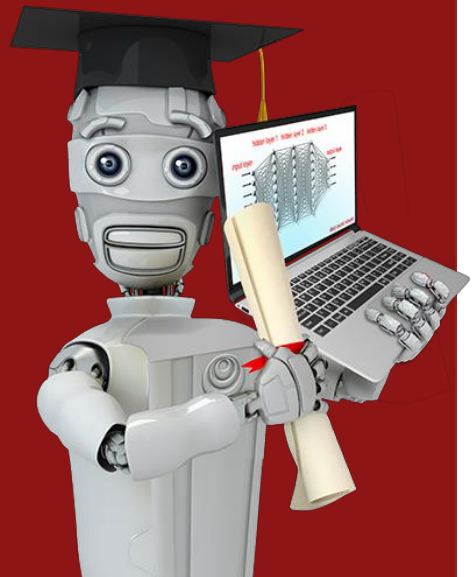
```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

fit

```
model.fit(X,Y,epochs=100)
```

DeepLearning.AI

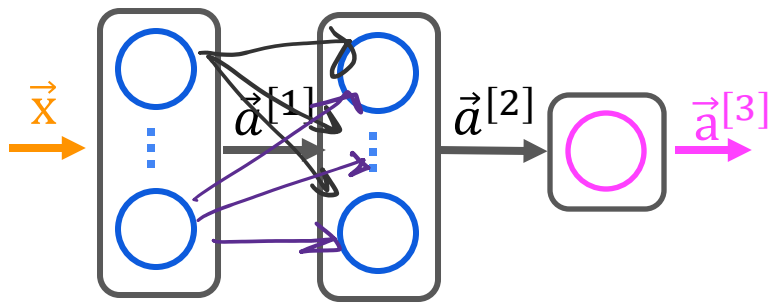
Stanford ONLINE



Additional Neural Network Concepts

Additional Layer Types

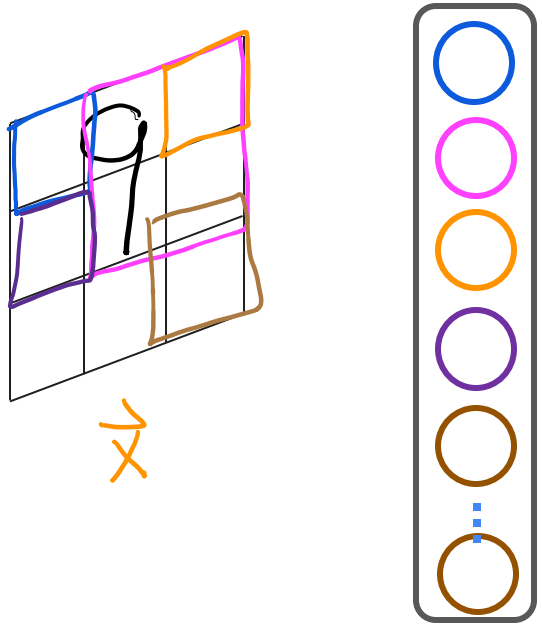
Dense Layer



Each neuron output is a function of all the activation outputs of the previous layer.

$$\vec{a}_1^{[2]} = g\left(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]}\right)$$

Convolutional Layer



Each Neuron only looks at part of the previous layer's inputs.

Why?

- Faster computation
- Need less training data (less prone to overfitting)

Convolutional Neural Network